ONLINE APPOINTMENT BOOKING SYSTEM FOR CLINICS

(Using Microsoft Azure | Agile – 2 Sprints)

THEME

Theme Title

Smart and Secure Online Appointment Booking System for Clinics

Theme Description

The Online Appointment Booking System is a cloud-based healthcare application designed to automate and simplify appointment scheduling between patients and doctors. Patients can securely register, search doctors by specialization, and book available time slots in real time. Doctors can manage profiles, define schedules, and view upcoming appointments. The system ensures secure handling of sensitive patient data, reliable time-slot validation, scalability during peak usage, and automated notifications. The solution follows **Agile methodology** and is deployed on the **Microsoft Azure platform** for high availability, security, and performance.

EPICS, FEATURES, USER STORIES, TASKS & BUGS

EPIC 1: User Authentication & Role Management

Feature 1.1: Patient / Doctor / Admin Registration

User Story 1.1a

As a patient, I want to register using strong credentials (minimum 8 characters, 1 number, 1 special character), so that my medical data remains secure.

Tasks / Bugs 1.1a

- Design registration UI
- Integrate **Azure Active Directory B2C**
- Implement password policy enforcement
- Encrypt credentials
- Store user profile in **Azure SQL Database**

Bugs

- Weak password accepted
- Duplicate email registration allowed

User Story 1.1b

As a user, I want to receive an email verification link after registration, so that my account is authenticated.

Tasks / Bugs 1.1b

- Configure email service using **Azure Communication Services**
- Generate verification token
- Set token expiry (24 hours)
- Activate account upon verification

Bugs

- Verification link expires immediately
- Email not delivered

Feature 1.2: Login / Logout

User Story 1.2a

As a user, I want to log in within 5 seconds, so that I can quickly access the system.

Tasks / Bugs 1.2a

- Implement JWT authentication
- Optimize login queries
- Enable session handling
- Monitor response time using **Azure Monitor**

Bugs

- Login exceeds 8 seconds during peak hours
- Session not cleared after logout

EPIC 2: Doctor Profile & Schedule Management

Feature 2.1: Doctor Profile Creation

User Story 2.1

As a doctor, I want to create a profile with specialization and experience, so that patients can choose the right doctor.

Tasks / Bugs 2.1

- Design doctor profile UI
- Store specialization details in database

Bugs

- Profile updates not saved
- Incorrect specialization mapping

Feature 2.2: Availability & Time-Slot Management

User Story 2.2

As a doctor, I want to define my available time slots, so that appointments are scheduled without conflicts.

Tasks / Bugs 2.2

- Time-slot calendar module
- Store availability in Azure SQL
- Validate overlapping slots

Bugs

- Overlapping time slots allowed
- Availability not updated in real time

EPIC 3: Appointment Booking & Time-Slot Handling

Feature 3.1: Appointment Booking

User Story 3.1

As a patient, I want to book appointments based on specialization and availability, so that I get timely consultation.

Tasks / Bugs 3.1

- Search doctors by specialization
- Check real-time availability
- Confirm booking

Bugs

- Double booking allowed
- Appointment not reflected in doctor calendar

Feature 3.2: Appointment Update / Cancellation

User Story 3.2

As a patient, I want to reschedule or cancel appointments, so that plans remain flexible.

Tasks / Bugs 3.2

- Appointment modification logic
- Cancellation confirmation

Bugs

- Cancellation not freeing time slot
- Updates not reflected immediately

EPIC 4: Notifications & Alerts

Feature 4.1: Automated Notifications

User Story 4.1

As a patient, I want appointment reminders, so that I don’t miss consultations.

Tasks / Bugs 4.1

- Trigger notifications via **Azure Functions**
- Schedule reminders

Bugs

- Reminder not sent
- Incorrect appointment time in notification

EPIC 5: Feedback & Consultation History

Feature 5.1: Feedback Submission

User Story 5.1

As a patient, I want to submit feedback after consultation, so that service quality improves.

Tasks / Bugs 5.1

- Feedback form
- Secure storage

Bugs

- Feedback not saved
- Duplicate submissions allowed

Feature 5.2: Appointment History

User Story 5.2

As a patient, I want to view my consultation history, so that I can track my medical visits.

Tasks / Bugs 5.2

- History retrieval module

Bugs

- Incomplete records displayed

EPIC 6: Security, Reliability & Monitoring

Feature 6.1: Non-Functional Requirements

User Story 6.1

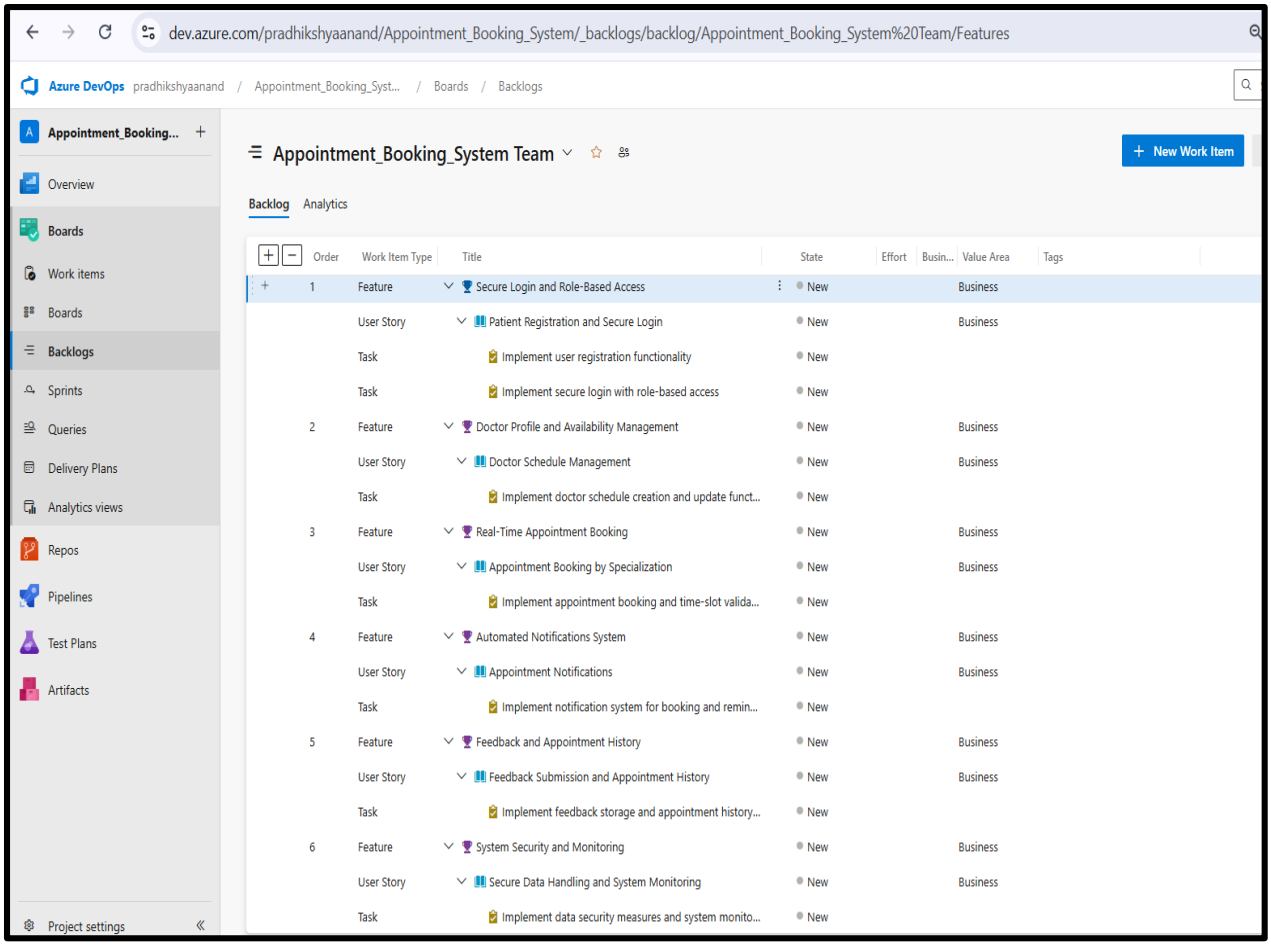
As an admin, I want the system to be reliable, scalable, and secure, so that patient data is protected.

Tasks / Bugs 6.1

- Enable HTTPS & encryption
- Configure auto-scaling on **Azure App Service**
- Monitor logs and alerts

Bugs

- System slowdown during peak load



AGILE PLANNING (2 SPRINTS)

Team Structure (4 Members)

- **Scrum Master** – Sprint planning, stand-ups
- **Backend Developer** – APIs, database, Azure Functions
- **Frontend Developer** – UI/UX
- **QA/Test Engineer** – Test cases, Azure Test Plans

Sprint 1 (Week 1–2): Core Features

Goal: Functional foundation

- User registration & login
- Doctor profile & schedule
- Appointment booking
- Azure App Service deployment

Sprint 2 (Week 3–4): Advanced Features & Quality

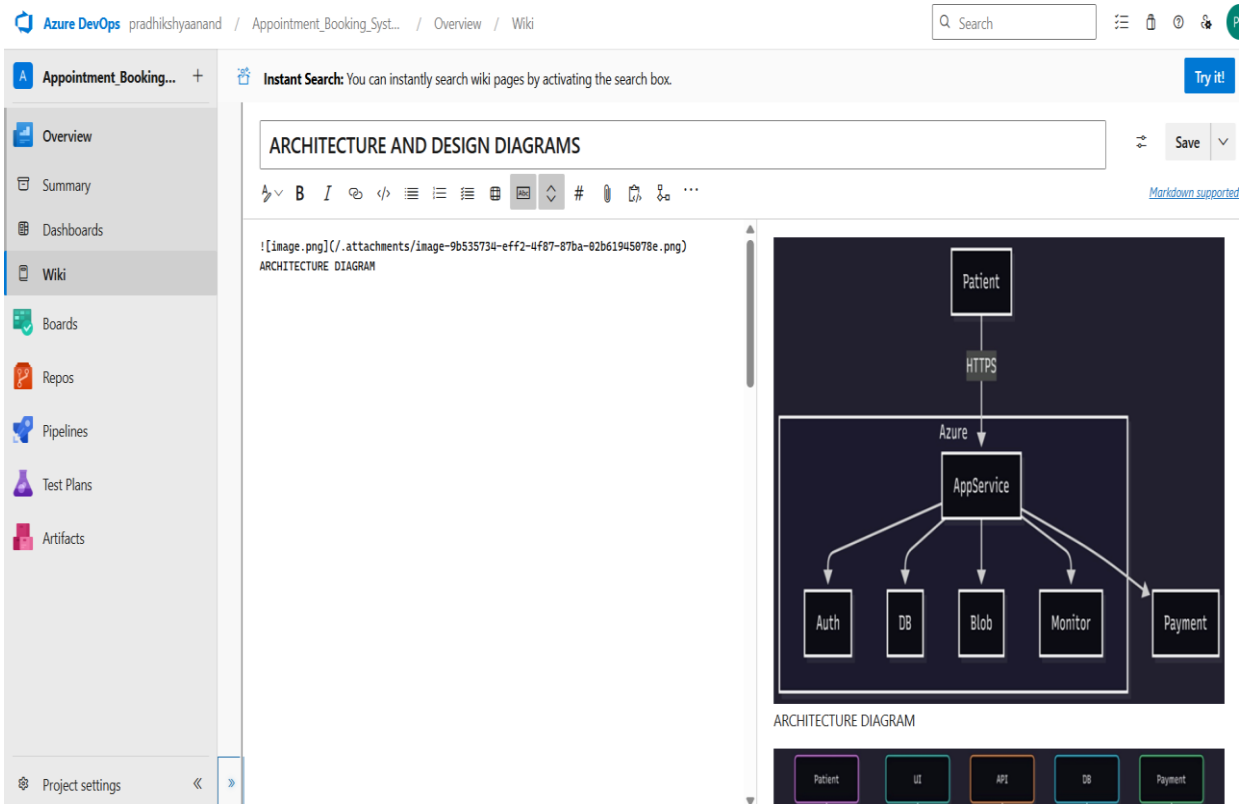
Goal: Reliability & optimization

- Notifications
- Feedback & history
- Load testing
- CI/CD pipeline
- Bug fixing

ARCHITECTURE & DESIGN DIAGRAMS (Azure)

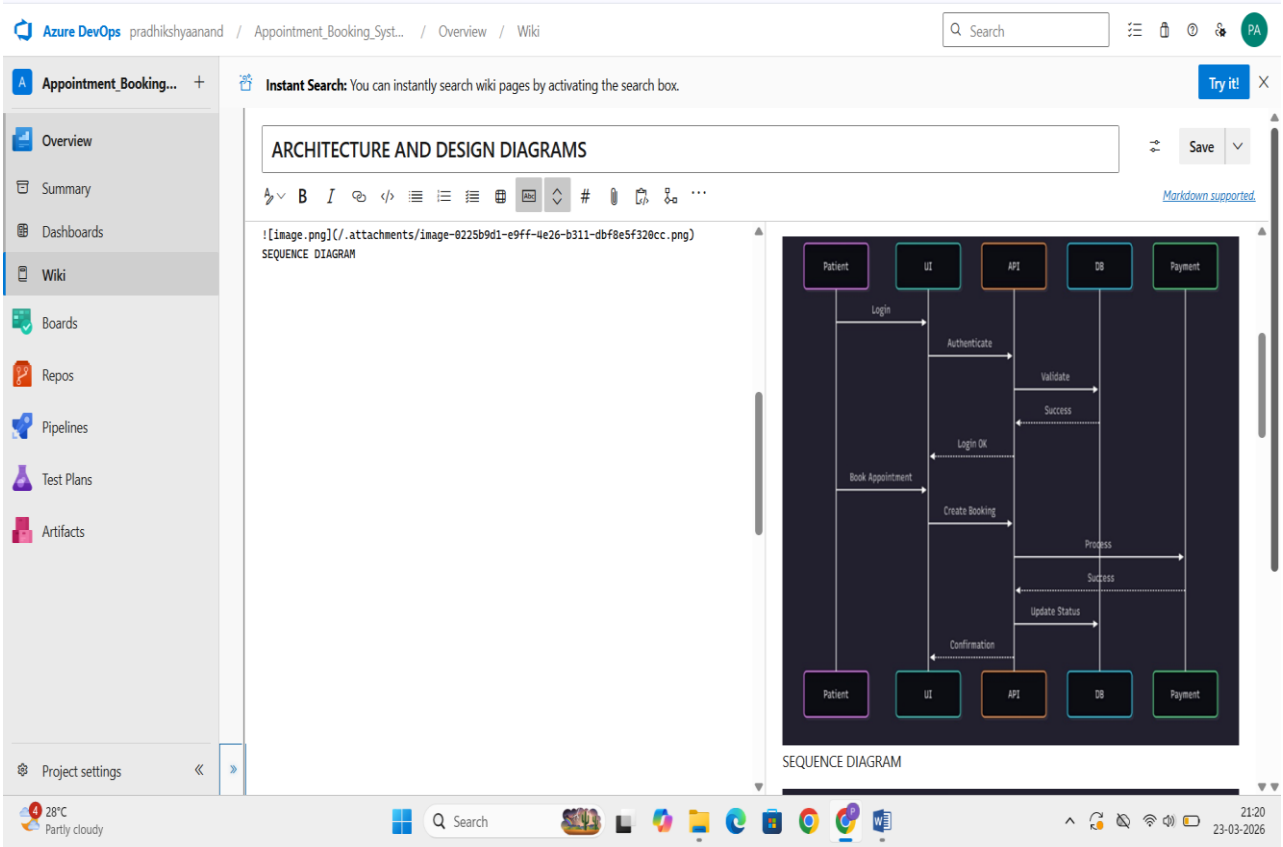
Architecture Diagram

```
Mermaid

flowchart TD

%% -------------------- PATIENT --------------------
A[Patient User] --> B[Frontend UI (Web/App)]

%% -------------------- AUTH --------------------
B --> C{Login / Signup}
C -->|New User| D[Register]
C -->|Existing User| E[Login]

D --> F[Patient Dashboard]
E --> F

%% -------------------- SEARCH & SELECT --------------------
F --> G[Search Doctor / Specialization]

G --> H[View Doctor Profile]

H --> I[Select Date & Time Slot]

%% -------------------- BACKEND PROCESS --------------------
I --> J[Send Request to Backend API]

J --> K{Check Slot Availability}

K -->|Available| L[Proceed Booking]
K -->|Not Available| I

%% -------------------- BOOKING --------------------
L --> M[Enter Symptoms (Optional)]

M --> N{Payment Required?}
```



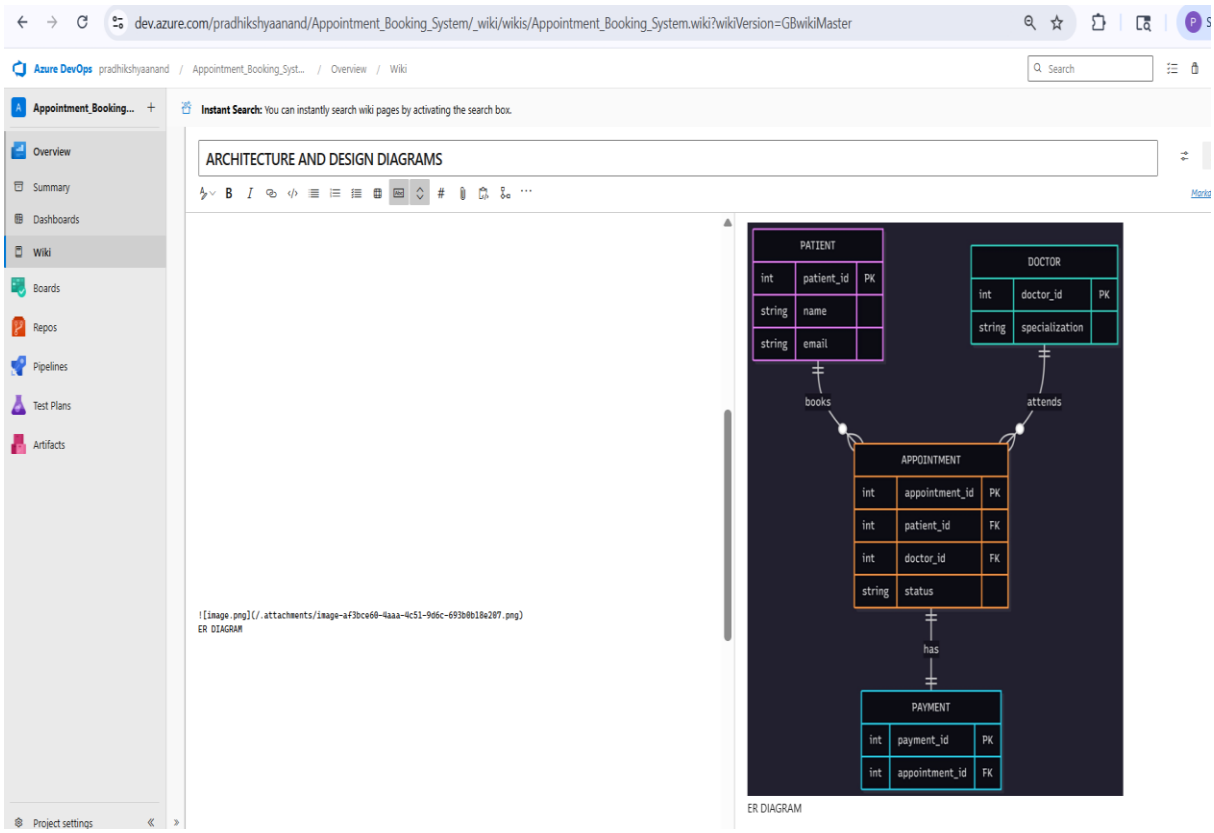
Sequence Diagram

```
Mermaid

sequenceDiagram
    participant P as Patient
    participant UI as Frontend (Web/App)
    participant API as Backend Server (API)
    participant DB as Database
    participant PAY as Payment Gateway
    participant NS as Notification Service
    participant AI as AI Recommendation (Optional)

    %% -------------------- LOGIN --------------------
    P->>UI: Open App / Website
    UI->>API: Login / Signup Request
    API->>DB: Validate / Store User
    DB-->>API: User डेटा
    API-->>UI: Auth Success
    UI-->>P: Show Dashboard

    %% -------------------- AI RECOMMENDATION --------------------
    UI->>AI: Request Doctor Suggestions
    AI-->>UI: Recommended Doctors

    %% -------------------- SEARCH DOCTOR --------------------
    P->>UI: Search Doctor / Specialization
    UI->>API: Fetch Doctor List
    API->>DB: Query Doctors
    DB-->>API: Doctor Data
    API-->>UI: Doctor List
    UI-->>P: Display Doctors

    %% -------------------- SELECT SLOT --------------------
    P->>UI: Select Date & Time Slot
    UI->>API: Check Slot Availability
```



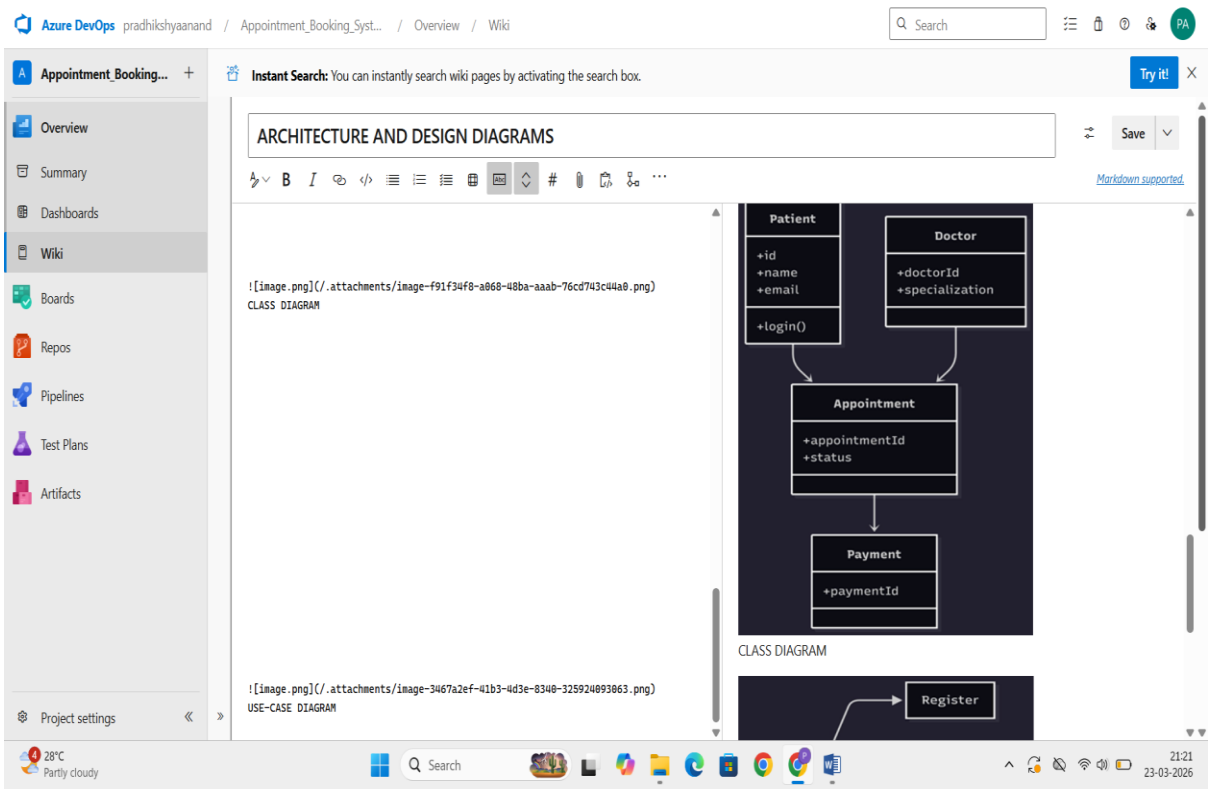
ER Diagram

```
%% ------------------- ENTITIES -------------------

PATIENT {
    int patient_id PK
    string name
    string email
    string phone
    string password
}

DOCTOR {
    int doctor_id PK
    string name
    string specialization
    float consultation_fee
}

APPOINTMENT {
    int appointment_id PK
    date appointment_date
    time appointment_time
    string status
    int patient_id FK
    int doctor_id FK
}

PAYMENT {
    int payment_id PK
    float amount
    string payment_status
    int appointment_id FK
}
```



Class Diagram

```mermaid
classDiagram

%% -------------------- MAIN CLASSES --------------------

class Patient {
    +int patientId
    +string name
    +string email
    +string phone
    +string password
    +login()
    +register()
    +searchDoctor()
    +bookAppointment()
    +cancelAppointment()
}

class Doctor {
    +int doctorId
    +string name
    +string specialization
    +float consultationFee
    +viewAvailability()
}

class Appointment {
    +int appointmentId
    +date appointmentDate
    +time appointmentTime
    +string status
    +createAppointment()
    +updateStatus()
}
```



Use Case Diagram

```
‹/› Mermaid

flowchart LR

%% -------------------- ACTOR --------------------
P([Patient])

%% -------------------- SYSTEM --------------------
subgraph System [Online Clinic Appointment System]

UC1((Register))
UC2((Login))
UC3((Search Doctor))
UC4((View Doctor Profile))
UC5((Select Time Slot))
UC6((Book Appointment))
UC7((Make Payment))
UC8((Receive Notification))
UC9((View Appointment))
UC10((Cancel / Reschedule))

end

%% -------------------- RELATIONSHIPS --------------------
P --> UC1
P --> UC2
P --> UC3
P --> UC4
P --> UC5
P --> UC6
P --> UC7
P --> UC8
P --> UC9
P --> UC10
```